

Single-layer NN/Shallow NN

Regression and Classification

Fundamentals of Deep Learning · Class 5 · Regression

3 August

Outline

Regression with Fixed Features

An Example Shallow Network

Generalising the Network

Universal Approximation

Training

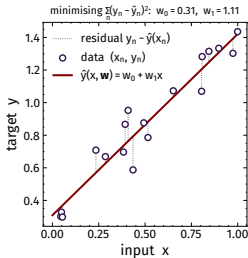
Regression with Fixed Features

Notation used throughout this course

$\mathbf{x} \in \mathbb{R}^D$	input / features	$\phi_j(\mathbf{x})$	fixed basis function
$y, \mathbf{y}$	target , targets	Φ	design matrix, $\Phi_{nj} = \phi_j(\mathbf{x}_n)$
$\hat{y}(\mathbf{x}, \mathbf{w})$	model prediction	a_j	pre-activation
N	number of examples	$h(\cdot)$	activation function
D, M, K	inputs, hidden units, outputs	$z_j = h(a_j)$	hidden unit
$\mathbf{w}, w_0$	weights, bias	$E(\mathbf{w})$	error function

Symbols follow **Bishop & Bishop (2024)**, except that targets are written y and predictions $\hat{y}$.

Linear regression, two views



$$\hat{y}(x, \mathbf{w}) = w_0 + w_1 x$$

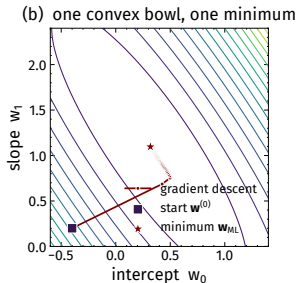
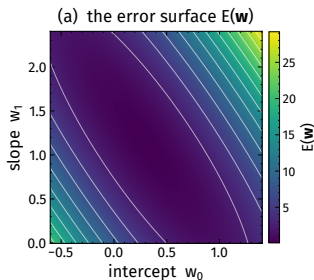
- Two parameters: intercept w_0 , slope w_1
- Fit by minimising the **sum-of-squares error**
- A **family** of lines; $\mathbf{w}$ selects one member

$N = 18$ noisy points; dotted segments are the residuals that get squared and summed.

ML model = family of functions

NN parameters select one member — this framing carries through the whole course

Training: minimise the error



(a) the error surface over the two parameters. (b) its contours, with gradient descent from a deliberately poor start to $\mathbf{w}_{ML}$.

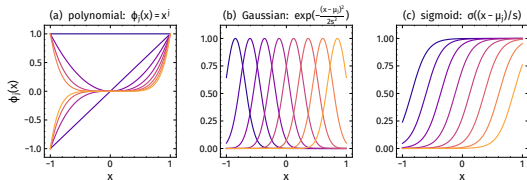
$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (\hat{y}(x_n, \mathbf{w}) - y_n)^2$$

- Each point of (w_0, w_1) space is one candidate line
- Walk downhill: **gradient descent**
- For a model **linear in $\mathbf{w}$** this surface is convex — one minimum, and a closed form

Beyond lines: fixed basis functions

$$\hat{y}(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^{M-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}) \quad (\text{B\&B 4.3})$$

Chosen **before** seeing data; only $\mathbf{w}$ is learned — still **linear** in $\mathbf{w}$, nonlinear in $\mathbf{x}$.



Three classical families, eight members each.

Result: the normal equations

Theorem 1 (Least-squares solution)

Let $\Phi \in \mathbb{R}^{N \times M}$ have $\Phi_{nj} = \phi_j(\mathbf{x}_n)$ and let $E(\mathbf{w}) = \frac{1}{2} \|\Phi \mathbf{w} - \mathbf{y}\|^2$. If Φ has full column rank, E has the unique minimiser

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y} = \Phi^\dagger \mathbf{y}. \quad (\text{B\&B 4.14})$$

- $\nabla_{\mathbf{w}} E = \Phi^\top (\Phi \mathbf{w} - \mathbf{y})$ (differentiate the quadratic)
- Set it to 0: $\Phi^\top \Phi \mathbf{w} = \Phi^\top \mathbf{y}$ — the normal equations
- $\nabla_{\mathbf{w}}^2 E = \Phi^\top \Phi \succ 0$ under full rank, so the stationary point is the global minimum
- Invert: $\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y}$



Result: the normal equations

Theorem 1 (Least-squares solution)

Let $\Phi \in \mathbb{R}^{N \times M}$ have $\Phi_{nj} = \phi_j(\mathbf{x}_n)$ and let $E(\mathbf{w}) = \frac{1}{2} \|\Phi \mathbf{w} - \mathbf{y}\|^2$. If Φ has full column rank, E has the unique minimiser

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y} = \Phi^\dagger \mathbf{y}. \quad (\text{B\&B 4.14})$$

- $\nabla_{\mathbf{w}} E = \Phi^\top (\Phi \mathbf{w} - \mathbf{y})$ (differentiate the quadratic)
- **Set it to 0: $\Phi^\top \Phi \mathbf{w} = \Phi^\top \mathbf{y}$ — the normal equations**
- $\nabla_{\mathbf{w}}^2 E = \Phi^\top \Phi \succ 0$ under full rank, so the stationary point is the global minimum
- Invert: $\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y}$



Result: the normal equations

Theorem 1 (Least-squares solution)

Let $\Phi \in \mathbb{R}^{N \times M}$ have $\Phi_{nj} = \phi_j(\mathbf{x}_n)$ and let $E(\mathbf{w}) = \frac{1}{2} \|\Phi \mathbf{w} - \mathbf{y}\|^2$. If Φ has full column rank, E has the unique minimiser

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y} = \Phi^\dagger \mathbf{y}. \quad (\text{B\&B 4.14})$$

- $\nabla_{\mathbf{w}} E = \Phi^\top (\Phi \mathbf{w} - \mathbf{y})$ (differentiate the quadratic)
- Set it to 0: $\Phi^\top \Phi \mathbf{w} = \Phi^\top \mathbf{y}$ — the normal equations
- $\nabla_{\mathbf{w}}^2 E = \Phi^\top \Phi \succ 0$ under full rank, so the stationary point is the global minimum
- Invert: $\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y}$



Result: the normal equations

Theorem 1 (Least-squares solution)

Let $\Phi \in \mathbb{R}^{N \times M}$ have $\Phi_{nj} = \phi_j(\mathbf{x}_n)$ and let $E(\mathbf{w}) = \frac{1}{2} \|\Phi \mathbf{w} - \mathbf{y}\|^2$. If Φ has full column rank, E has the unique minimiser

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y} = \Phi^\dagger \mathbf{y}. \quad (\text{B\&B 4.14})$$

- $\nabla_{\mathbf{w}} E = \Phi^\top (\Phi \mathbf{w} - \mathbf{y})$ (differentiate the quadratic)
- Set it to 0: $\Phi^\top \Phi \mathbf{w} = \Phi^\top \mathbf{y}$ — the **normal equations**
- $\nabla_{\mathbf{w}}^2 E = \Phi^\top \Phi \succ 0$ under full rank, so the stationary point is the global minimum
- **Invert:** $\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y}$



Least squares in closed form: the geometry

Proposition 2 (Orthogonal projection)

$\hat{\mathbf{y}} = \Phi \mathbf{w}_{\text{ML}}$ is the orthogonal projection of $\mathbf{y}$ onto $\mathcal{S} = \text{span}\{\boldsymbol{\varphi}_1, \dots, \boldsymbol{\varphi}_M\}$.

Proof. The normal equations read $\Phi^T(\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{0}$, i.e. the residual is orthogonal to every column $\boldsymbol{\varphi}_j$, hence to all of $\mathcal{S}$. $\square$

- Columns $\boldsymbol{\varphi}_j \in \mathbb{R}^N$ span an M -dimensional subspace of $\mathbb{R}^N$
- Least squares = drop a perpendicular

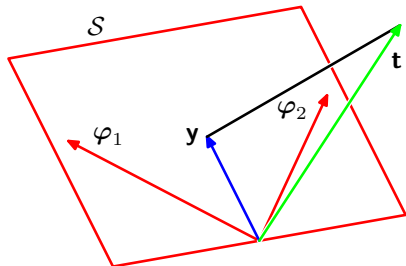


Figure from Bishop & Bishop (2024), Fig. 4.3. $\mathbf{t}$ there is our target vector $\mathbf{y}$.

ML closed form exists

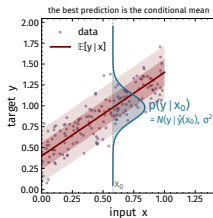
NN parameters inside $h(\cdot)$ destroy it

Why least squares? The probabilistic view

Proposition 3 (Least squares = maximum likelihood)

Under $p(y | x, \mathbf{w}, \sigma^2) = \mathcal{N}(y | \hat{y}(x, \mathbf{w}), \sigma^2)$ with i.i.d. data, maximising the likelihood in $\mathbf{w}$ is equivalent to minimising $\sum_n (y_n - \hat{y}_n)^2$.

Proof. $\ln p(\mathbf{y} | \mathbf{w}, \sigma^2) = -\frac{1}{2\sigma^2} \sum_n (y_n - \hat{y}_n)^2 - \frac{N}{2} \ln \sigma^2 - \frac{N}{2} \ln 2\pi$. Only the first term involves $\mathbf{w}$, and $\sigma^2 > 0$, so the two problems have the same argmin. $\square$



Bands are ± 1 and ± 2 standard deviations; the turquoise curve is $p(y | x_0)$ at one input.

ML Gaussian noise $\rightarrow$ squared error

NN Class 6: Bernoulli noise $\rightarrow$ cross-entropy

What is the best possible prediction?

Theorem 4 (The optimal predictor is the conditional mean)

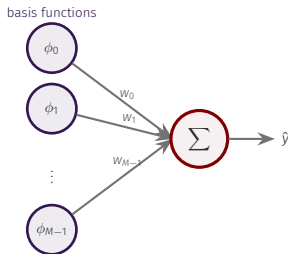
For squared loss, the function minimising $\mathbb{E}_{\mathbf{x},y}[(f(\mathbf{x}) - y)^2]$ over all measurable f is $f^*(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}]$, and for any f ,

$$\mathbb{E}[(f - y)^2] = \underbrace{\int (f(\mathbf{x}) - f^*(\mathbf{x}))^2 p(\mathbf{x}) d\mathbf{x}}_{\text{what the model controls}} + \underbrace{\int \text{var}[y \mid \mathbf{x}] p(\mathbf{x}) d\mathbf{x}}_{\text{intrinsic noise}}.$$

Proof. Write $f - y = (f - f^*) + (f^* - y)$ and expand. The cross term vanishes because $\mathbb{E}[f^*(\mathbf{x}) - y \mid \mathbf{x}] = 0$ by definition of f^* ; both remaining terms are non-negative and the second is free of f . □ B&B, 2024, §4.2

The noise floor is **irreducible**. All a network can do is close the first term — which is exactly why we care how rich its function family is.

Linear regression is itself a network



One node per basis function, one weight per link, summed at the output.

- Nodes: $\phi_j(\mathbf{x})$; links: weights w_j
- Already a one-layer network — but the ϕ_j carry no parameters

The limitation

For images, audio and text nobody knows how to write good ϕ_j by hand.

ML ϕ_j fixed by design

NN next: give the basis functions *parameters*

Why shallow networks

A linear model in fixed features is limited. We want a model that is:

- flexible enough to describe **arbitrarily complex** input–output mappings,
- able to take **as many inputs** as we want,
- able to produce **as many outputs** as we want.

Shallow neural networks do all three — by putting parameters *inside* the basis functions and learning them from data.

An Example Shallow Network

The model

Scalar input x , scalar output, $M = 3$ hidden units, ten parameters (Bishop & Bishop, 2024, Eqs. 6.1–6.3):

$$a_j = w_{j0}^{(1)} + w_{j1}^{(1)}x, \quad z_j = h(a_j), \quad \hat{y} = w_0^{(2)} + \sum_{j=1}^M w_j^{(2)}z_j$$

- Compare $\hat{y} = \sum_j w_j \phi_j(\mathbf{x})$: the same outer form, but the features z_j now **contain parameters**
- Given $\mathbf{w}$: inference = evaluate the three equations
- Given data: same sum-of-squares $E(\mathbf{w})$, minimised over *all* the weights

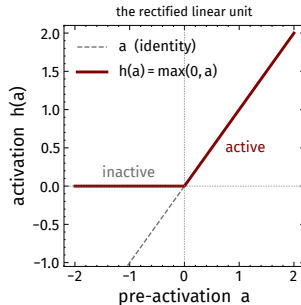
ML $\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})$: linear in *fixed* features

NN $\sum_j w_j^{(2)} z_j$: linear in *parameterised* features

The activation function: ReLU

$$h(a) = \text{ReLU}(a) = \max(0, a) = \begin{cases} 0 & a < 0 \\ a & a \geq 0 \end{cases}$$

- The **rectified linear unit**: clips negative pre-activations to zero
- Without it the model collapses:
 $\sum_j w_j^{(2)}(w_{j0}^{(1)} + w_{j1}^{(1)}x)$ is just a line again



Positive pre-activations pass through unchanged;
everything negative is clipped to zero.

ML fixed nonlinearity: x^j , Gaussian, sigmoid

NN nonlinearity applied to a *learned* line

Hidden units: the model in two stages

Stage 1 — three linear functions, each passed through h :

$$z_1 = h(w_{10}^{(1)} + w_{11}^{(1)}x), \quad z_2 = h(w_{20}^{(1)} + w_{21}^{(1)}x), \quad z_3 = h(w_{30}^{(1)} + w_{31}^{(1)}x)$$

Stage 2 — weight and sum:

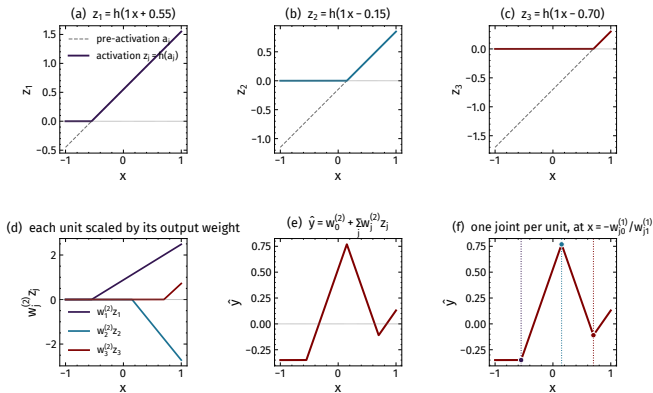
$$\hat{y} = w_0^{(2)} + w_1^{(2)}z_1 + w_2^{(2)}z_2 + w_3^{(2)}z_3$$

- z_1, z_2, z_3 are the **hidden units**
- Clipped $\rightarrow$ **inactive**; passing its input $\rightarrow$ **active**

ML regress y on fixed $\phi(x)$

NN Stage 2 is that regression, on learned z

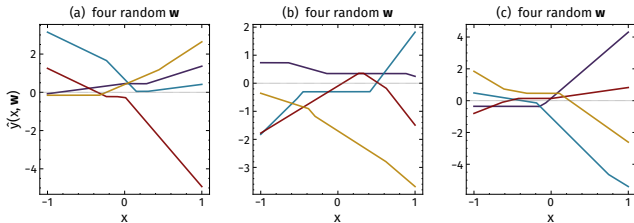
Computing the function, step by step



(a–c) one unit each: dashed a_j , solid $z_j = h(a_j)$. (d) scaled by $w_j^{(2)}$. (e) summed, plus $w_0^{(2)}$. (f) the joints.

The function family: piecewise linear

$M = 3$ units $\Rightarrow$ continuous, piecewise linear, at most $M + 1 = 4$ pieces



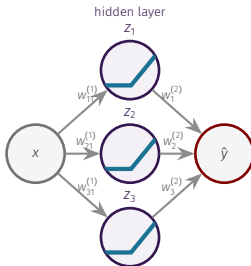
Twelve networks with $M = 3$ units and randomly drawn weights: all continuous and piecewise linear, differing only in where the joints fall and how steep each piece is.

- Continuous piecewise-linear functions
- One **joint** per ReLU, at $x = -w_{j0}^{(1)} / w_{j1}^{(1)}$
- Each region $\leftrightarrow$ an **activation pattern**: which units are on

ML family: span of the chosen ϕ_j

NN family: piecewise linear, joints placed by training

Depicting the network

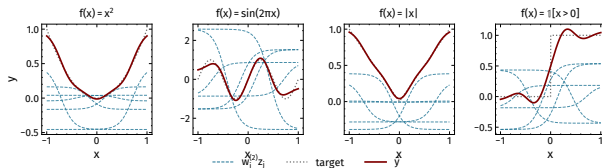


Each arrow carries one weight; the turquoise glyph inside a unit marks where the activation h is applied. Biases $w_{j0}^{(1)}$ and $w_0^{(2)}$ are not drawn.

- Each **weight multiplies its source and adds to its target**
- Ten parameters: six into the hidden layer, four out of it

Hidden units are basis functions that adapt

same architecture ($M = 6, \tanh$), four targets: the units rearrange themselves



Dashed: the individual weighted hidden units $w_j^{(2)} z_j$ after fitting. Dotted: the target. Solid: the network output. The same architecture rearranges its units differently for each target.

- Same M , same activation — but the effective basis is **chosen by the data**
- This is the whole difference from $\hat{y} = \mathbf{w}^\top \phi(\mathbf{x})$

Generalising the Network

Capacity: M hidden units

$$z_j = h(w_{j0}^{(1)} + w_{j1}^{(1)}x), \quad \hat{y} = w_0^{(2)} + \sum_{j=1}^M w_j^{(2)}z_j$$

- The number of hidden units M is a measure of **network capacity**
- M ReLU units on a scalar input give M joints, hence at most **$M+1$ linear regions**
- Parameter count for D inputs, M units, K outputs: $M(D+1) + K(M+1)$

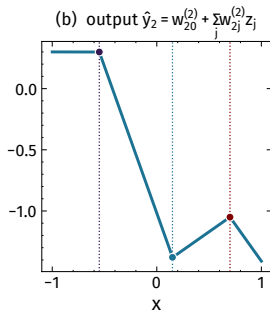
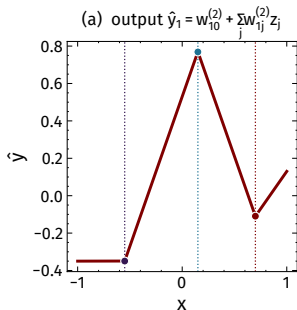
ML capacity dial: number of basis functions M

NN capacity dial: number of hidden units M

More than one output

shared hidden units $\Rightarrow$ joints at the same x

$D = 1, M = 4, K = 2$:

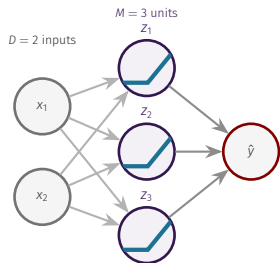


$$\hat{y}_k = w_{k0}^{(2)} + \sum_{j=1}^M w_{kj}^{(2)} z_j$$

- One extra linear read-out per output; the hidden units are **shared**
- Hence the joints sit at the **same places**

Coloured markers are the joints. Both outputs bend at exactly the same x , because they read the same z_j .

More than one input



Every hidden unit now receives *both* inputs, so
each has two slopes and one bias.

$$a_j = w_{j0}^{(1)} + w_{j1}^{(1)}x_1 + w_{j2}^{(1)}x_2, \quad z_j = h(a_j)$$

$$\hat{y} = w_0^{(2)} + \sum_{j=1}^3 w_j^{(2)}z_j$$

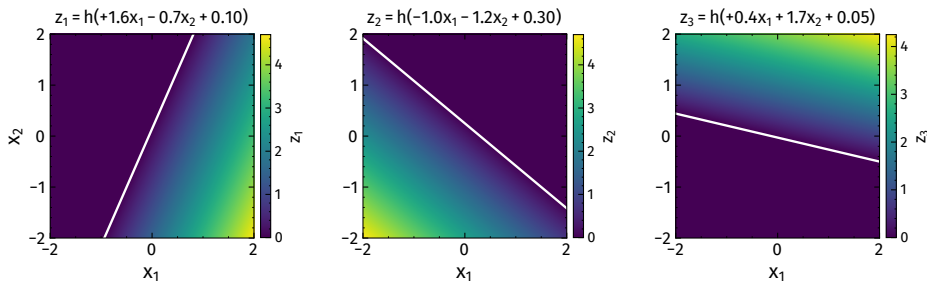
- Each pre-activation is an **oriented plane** over the input space
- $a_j = 0$ is a **line**: on one side the unit is off, on the other it rises linearly

ML $\phi_j : \mathbb{R}^D \rightarrow \mathbb{R}$, hand-designed

NN $z_j : \mathbb{R}^D \rightarrow \mathbb{R}$, an oriented hinge, learned

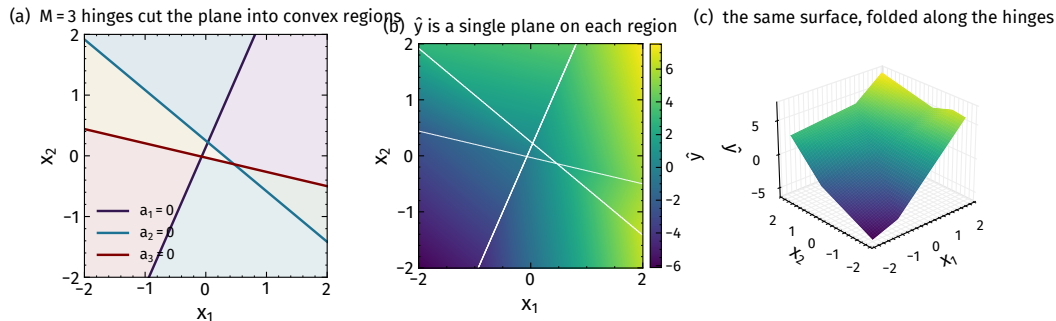
Each unit is a hinge over the input plane

white line: $a_j = 0$, where the unit switches on



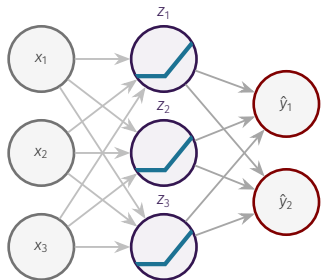
The three hidden units of that network, drawn over (x_1, x_2) . Each is flat (dark) on one side of its white line $a_j = 0$ and rises linearly on the other; the orientation and offset of the line are set by $w_{j1}^{(1)}$, $w_{j2}^{(1)}$ and $w_{j0}^{(1)}$.

Two inputs, computed step by step



(a) the three lines $a_j = 0$ cut the plane into **convex regions**, one per activation pattern. (b) $\hat{y}$ is a single plane on each region, with the borders in white. (c) the same output as a landscape — one surface, folded along the hinge lines.

The general case



$D = 3$ inputs, $M = 3$ hidden units, $K = 2$ outputs –
 $3(3+1) + 2(3+1) = 20$ parameters.

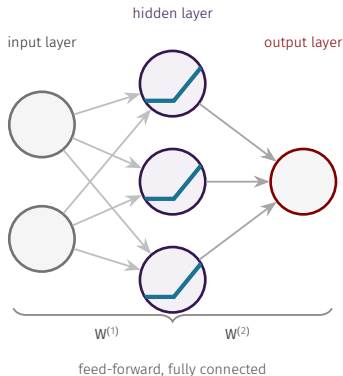
D inputs, M hidden units, K outputs:

$$z_j = h\left(\sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) \quad (\text{B\&B 6.1--6.2})$$

$$\hat{y}_k = \sum_{j=1}^M w_{kj}^{(2)} z_j + w_{k0}^{(2)} \quad (\text{B\&B 6.3})$$

- In matrix form: $\hat{\mathbf{y}} = \mathbf{W}^{(2)} h(\mathbf{W}^{(1)} \mathbf{x})$ with biases absorbed as $x_0 = 1$, $z_0 = 1$

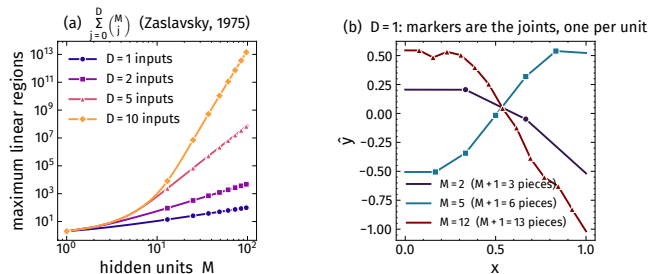
Terminology



The standard vocabulary for the parts of a network.

- input / hidden / output layer
- weights (slopes) and biases (offsets)
- pre-activations a_j , activations z_j
- fully connected, feed-forward; one hidden layer = shallow
- with one or more hidden layers, also called a multi-layer perceptron

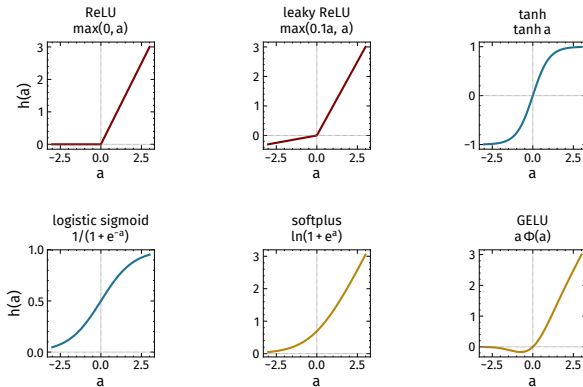
How many regions?



(a) the bound against the number of units, for several input dimensions D (log-log). (b) three networks of different width on a scalar input; markers are the joints.

- **Zaslavsky (1975)**: M hyperplanes in general position in $\mathbb{R}^D$ create at most $\sum_{j=0}^D \binom{M}{j}$ regions
- $D = 10, M = 100$ units (≈ 1200 parameters) already exceeds 10^{13} regions

Other activation functions



ReLU is the modern default; **tanh** and the logistic sigmoid were historically standard; softplus and GELU are smooth relatives of ReLU. Only the *shape* changes — the architecture does not.

Universal Approximation

The question

With enough hidden units, how much can one hidden layer express?

- Every function the network computes is **continuous and piecewise linear** (with ReLU)
- Adding a unit adds a joint — one more piece to play with
- So the question is whether piecewise-linear functions can get **arbitrarily close** to an arbitrary continuous target

The answer is yes, and for $D = 1$ with ReLU we can prove it **constructively** — the proof tells us exactly which weights to use.

Universal approximation: the statement

Theorem 5 (Universal approximation; Cybenko 1989, Hornik 1991)

Let $h : \mathbb{R} \rightarrow \mathbb{R}$ be continuous and *not a polynomial*, let $\mathcal{K} \subset \mathbb{R}^D$ be compact, and let $f : \mathcal{K} \rightarrow \mathbb{R}$ be continuous. Then for every $\varepsilon > 0$ there exist a finite M and weights $\mathbf{w}_j^{(1)} \in \mathbb{R}^D$, $w_{j_0}^{(1)}, w_j^{(2)} \in \mathbb{R}$ such that

$$\hat{y}(\mathbf{x}) = w_0^{(2)} + \sum_{j=1}^M w_j^{(2)} h(\mathbf{w}_j^{(1)\top} \mathbf{x} + w_{j_0}^{(1)}) \quad \text{satisfies} \quad \sup_{\mathbf{x} \in \mathcal{K}} |f(\mathbf{x}) - \hat{y}(\mathbf{x})| < \varepsilon.$$

- One hidden layer is enough — *width*, not depth, is the resource being spent
- “Not a polynomial” is *necessary*: if $\deg h = p$, every network output is a polynomial of degree $\leq p$, and those are not dense in $C(\mathcal{K})$ (Leshno et al., 1993)

Lemma 1: two ReLUs make a ramp

Lemma 6 (Ramps and steps)

For $a \in \mathbb{R}$ and $\delta > 0$ define $g_\delta(x) = \frac{1}{\delta} [h(x - a) - h(x - a - \delta)]$ with $h = \text{ReLU}$. Then g_δ is continuous, $g_\delta \equiv 0$ for $x \leq a$, $g_\delta \equiv 1$ for $x \geq a + \delta$, and it is linear in between. Hence $g_\delta(x) \rightarrow \mathbb{1}_{[x > a]}$ pointwise as $\delta \downarrow 0$.

Proof. Check the three cases directly. For $x \leq a$ both ReLUs vanish. For $a < x < a + \delta$ the first is $x - a$ and the second still vanishes, giving $(x - a)/\delta$. For $x \geq a + \delta$ the difference is $(x - a) - (x - a - \delta) = \delta$, so $g_\delta = 1$. □

Two hidden units buy a **switch**. Everything else is bookkeeping: a sum of switches is a staircase, and a staircase with fine enough steps tracks any continuous function.

Lemma 2: the interpolant *is* a network

Lemma 7 (Exact representation of a piecewise-linear function)

Let $x_0 < x_1 < \dots < x_M$ and let $\hat{y}$ be the continuous function that is linear on each $[x_{j-1}, x_j]$ with slope s_j . Then for all $x \in [x_0, x_M]$

$$\hat{y}(x) = \hat{y}(x_0) + s_1 h(x - x_0) + \sum_{j=1}^{M-1} (s_{j+1} - s_j) h(x - x_j),$$

a shallow ReLU network with M hidden units whose output weights are the *slope increments*.

Proof. Both sides are continuous and piecewise linear with the same breakpoints, so it suffices to match value and slope. At $x = x_0$ every $h(\cdot)$ term vanishes, so both sides equal $\hat{y}(x_0)$. On (x_{k-1}, x_k) the active terms are those with $x_j < x$, so the right-hand slope telescopes to $s_1 + \sum_{j=1}^{k-1} (s_{j+1} - s_j) = s_k$. □

Proof of the theorem for $D = 1$, ReLU

Let f be continuous on $[0, 1]$ and fix $\varepsilon > 0$.

- **Step 1 (uniform continuity).** $[0, 1]$ is compact, so f is uniformly continuous: there is $\delta > 0$ with $|f(u) - f(v)| < \varepsilon/2$ whenever $|u - v| < \delta$.
- **Step 2 (choose the grid).** Take $M = \lceil 1/\delta \rceil$ and $x_j = j/M$, so the mesh $1/M < \delta$. Let $\hat{y}$ be the piecewise-linear interpolant of f at these knots.
- **Step 3 (bound the error).** For $x \in [x_{j-1}, x_j]$, $\hat{y}(x)$ is a convex combination $\lambda f(x_{j-1}) + (1 - \lambda)f(x_j)$, so $|\hat{y}(x) - f(x_{j-1})| \leq \max\{|f(x_{j-1}) - f(x_j)|, 0\} < \varepsilon/2$, giving $|f(x) - \hat{y}(x)| \leq |f(x) - f(x_{j-1})| + |f(x_{j-1}) - \hat{y}(x)| < \varepsilon$.
- **Step 4 (it is a network).** By Lemma 2, $\hat{y}$ is exactly a shallow ReLU network with M hidden units. Therefore $\sup_{x \in [0, 1]} |f(x) - \hat{y}(x)| < \varepsilon$. $\square$

Proof of the theorem for $D = 1$, ReLU

Let f be continuous on $[0, 1]$ and fix $\varepsilon > 0$.

- **Step 1 (uniform continuity).** $[0, 1]$ is compact, so f is uniformly continuous: there is $\delta > 0$ with $|f(u) - f(v)| < \varepsilon/2$ whenever $|u - v| < \delta$.
- **Step 2 (choose the grid).** Take $M = \lceil 1/\delta \rceil$ and $x_j = j/M$, so the mesh $1/M < \delta$. Let $\hat{y}$ be the piecewise-linear interpolant of f at these knots.
- **Step 3 (bound the error).** For $x \in [x_{j-1}, x_j]$, $\hat{y}(x)$ is a convex combination $\lambda f(x_{j-1}) + (1 - \lambda)f(x_j)$, so $|\hat{y}(x) - f(x_{j-1})| \leq \max\{|f(x_{j-1}) - f(x_j)|, 0\} < \varepsilon/2$, giving $|f(x) - \hat{y}(x)| \leq |f(x) - f(x_{j-1})| + |f(x_{j-1}) - \hat{y}(x)| < \varepsilon$.
- **Step 4 (it is a network).** By Lemma 2, $\hat{y}$ is exactly a shallow ReLU network with M hidden units. Therefore $\sup_{x \in [0, 1]} |f(x) - \hat{y}(x)| < \varepsilon$. $\square$

Proof of the theorem for $D = 1$, ReLU

Let f be continuous on $[0, 1]$ and fix $\varepsilon > 0$.

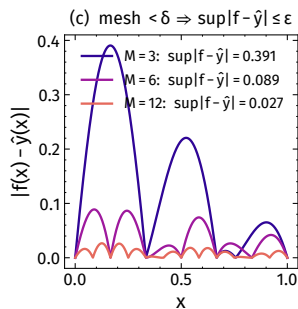
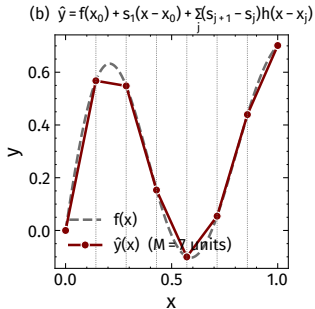
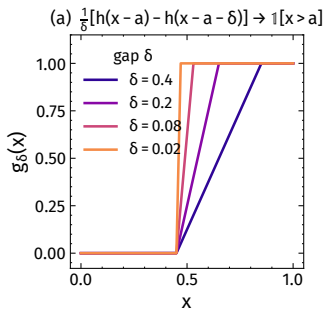
- **Step 1 (uniform continuity).** $[0, 1]$ is compact, so f is uniformly continuous: there is $\delta > 0$ with $|f(u) - f(v)| < \varepsilon/2$ whenever $|u - v| < \delta$.
- **Step 2 (choose the grid).** Take $M = \lceil 1/\delta \rceil$ and $x_j = j/M$, so the mesh $1/M < \delta$. Let $\hat{y}$ be the piecewise-linear interpolant of f at these knots.
- **Step 3 (bound the error).** For $x \in [x_{j-1}, x_j]$, $\hat{y}(x)$ is a convex combination $\lambda f(x_{j-1}) + (1 - \lambda)f(x_j)$, so $|\hat{y}(x) - f(x_{j-1})| \leq \max\{|f(x_{j-1}) - f(x_j)|, 0\} < \varepsilon/2$, giving $|f(x) - \hat{y}(x)| \leq |f(x) - f(x_{j-1})| + |f(x_{j-1}) - \hat{y}(x)| < \varepsilon$.
- **Step 4 (it is a network).** By Lemma 2, $\hat{y}$ is exactly a shallow ReLU network with M hidden units. Therefore $\sup_{x \in [0, 1]} |f(x) - \hat{y}(x)| < \varepsilon$. $\square$

Proof of the theorem for $D = 1$, ReLU

Let f be continuous on $[0, 1]$ and fix $\varepsilon > 0$.

- **Step 1 (uniform continuity).** $[0, 1]$ is compact, so f is uniformly continuous: there is $\delta > 0$ with $|f(u) - f(v)| < \varepsilon/2$ whenever $|u - v| < \delta$.
- **Step 2 (choose the grid).** Take $M = \lceil 1/\delta \rceil$ and $x_j = j/M$, so the mesh $1/M < \delta$. Let $\hat{y}$ be the piecewise-linear interpolant of f at these knots.
- **Step 3 (bound the error).** For $x \in [x_{j-1}, x_j]$, $\hat{y}(x)$ is a convex combination $\lambda f(x_{j-1}) + (1 - \lambda)f(x_j)$, so $|\hat{y}(x) - f(x_{j-1})| \leq \max\{|f(x_{j-1}) - f(x_j)|, 0\} < \varepsilon/2$, giving $|f(x) - \hat{y}(x)| \leq |f(x) - f(x_{j-1})| + |f(x_{j-1}) - \hat{y}(x)| < \varepsilon$.
- **Step 4 (it is a network).** By Lemma 2, $\hat{y}$ is exactly a shallow ReLU network with M hidden units. Therefore $\sup_{x \in [0, 1]} |f(x) - \hat{y}(x)| < \varepsilon$. □

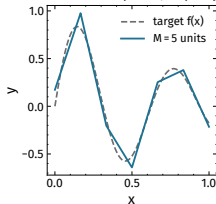
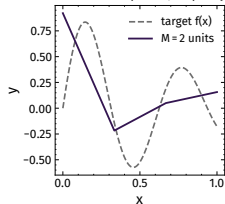
The construction, drawn



(a) Lemma 1: as the gap δ shrinks, the scaled ReLU difference becomes a unit step. (b) Lemma 2: the interpolant through the knots x_j is written exactly as a sum of ReLUs whose output weights are the slope increments. (c) Step 3: refining the mesh drives $\sup |f - \hat{y}|$ down.

Using the theorem: how fast does the error fall?

M=2: M+1=3 linear pieces, $\sup|f-\hat{y}| = 0.921$ M=5: M+1=6 linear pieces, $\sup|f-\hat{y}| = 0.173$

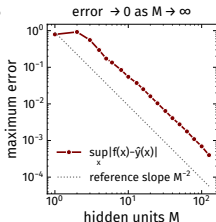
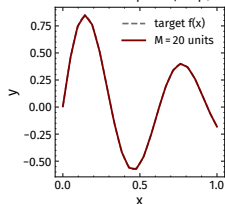


If f is Lipschitz with constant L on $[0, 1]$,
Step 1 lets us take $\delta = \varepsilon/(2L)$, so

$$M = \lceil 2L/\varepsilon \rceil$$

units suffice — **linear** in L/ε . If f is also $\mathcal{C}^2$, interpolation theory sharpens this to
 $\sup|f-\hat{y}| \leq \frac{1}{8}\|f''\|_{\infty}M^{-2}$.

M=20: M+1=21 linear pieces, $\sup|f-\hat{y}| = 0.016$



- Panels: $M = 2, 5, 20$ units on one target
- Last panel: the measured $\sup$ error, tracking the predicted M^{-2} slope

What the theorem does *not* say

- It is an **existence result**. It says good weights exist; it does not say gradient descent will find them, nor that they are learnable from finite data.
- It says **nothing about generalisation**. Fitting a training set arbitrarily well is not the goal — recall the irreducible noise term.
- The **width can be astronomical**. Repeating the grid argument in D dimensions needs $M = O((L/\varepsilon)^D)$ units: the **curse of dimensionality**.

This last point is the opening argument of Class 7: **depth** buys regions exponentially in the number of layers, where width buys them only polynomially.

Training

Same error, harder landscape

Sum-of-squares error, now with a network inside (Bishop & Bishop, 2024, §6.1):

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \|\hat{\mathbf{y}}(\mathbf{x}_n, \mathbf{w}) - \mathbf{y}_n\|^2$$

- Parameters sit *inside* $h(\cdot)$, so $\nabla E = \mathbf{0}$ has **no closed form** — the normal-equations theorem no longer applies
- The error surface is **non-convex**, unlike every model so far
- Training: gradient descent and backpropagation — Classes 11–13

ML normal equations: jump straight to the minimum

NN gradient descent: walk downhill, non-convex

Why the surface cannot be convex

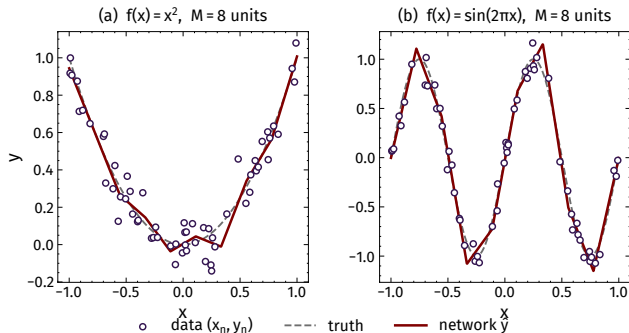
Proposition 8 (Weight-space symmetries; B&B §6.1.1)

*For a network with M hidden units and an odd activation ($h(-a) = -h(a)$, e.g. $\tanh$), every weight vector $\mathbf{w}$ has at least $M! 2^M$ distinct counterparts computing the *identical* function.*

Proof. (i) *Sign flip.* Replace $\mathbf{w}_j^{(1)} \rightarrow -\mathbf{w}_j^{(1)}$ and $w_{kj}^{(2)} \rightarrow -w_{kj}^{(2)}$. Then $w_{kj}^{(2)} h(a_j)$ is unchanged since h is odd. There are 2^M such choices. (ii) *Permutation.* Relabelling the hidden units permutes rows of $\mathbf{W}^{(1)}$ and columns of $\mathbf{W}^{(2)}$ together, leaving the sum unchanged: $M!$ choices. The two act independently. $\square$

Every minimum is repeated at least $M! 2^M$ times. A function with many separated global minima cannot be convex — the non-convexity is *structural*, not accidental.

It works: shallow networks fit real data



A shallow ReLU network with $M = 8$ units fitted by least squares to noisy samples of two functions. The learned units place their joints where the data bends; the fit tracks the truth without chasing every noisy point.

The dictionary

	Linear/Basis-Function Model	Shallow Neural Network
Function class	lines, polynomials, RBFs	piecewise-linear surfaces
Features	$\phi_j(\mathbf{x})$, fixed	$z_j = h(a_j)$, learned
Prediction	$\mathbf{w}^T \phi(\mathbf{x})$	$\sum_j w_{kj}^{(2)} z_j + w_{k0}^{(2)}$
Error (regression)	sum of squares	sum of squares
Optimisation	normal equations, convex	gradient descent, non-convex
Capacity	number of basis functions M	hidden units M , regions $\leq \sum_{j \leq D} \binom{M}{j}$

A shallow network is regression whose basis functions carry parameters and are learned from data.

Takeaways

1. A shallow network puts parameters inside the basis functions; the output layer is a **linear regression on learned features**
2. Least squares has a **closed form** and a clean geometry — an orthogonal projection — only while the model is linear in $\mathbf{w}$
3. With ReLU the family is **piecewise linear**: one joint per unit, regions counted by binomial coefficients
4. **Universal approximation** holds, and for $D = 1$ we proved it constructively — but it is existence, and the width needed grows like $(L/\varepsilon)^D$
5. The price: **no closed form**, a structurally non-convex error surface, gradient-based training

Class 6: the same architecture for **classification** — sigmoid/softmax outputs and cross-entropy error.

References

- C. M. Bishop & H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. §4.1 (basis functions, least squares, the geometry of the normal equations), §4.2 (decision theory for regression), §6.1 (feed-forward networks, weight-space symmetries). Figure 4.3 reproduced as printed.
- S. J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. Ch. 3 — shallow networks, the piecewise-linear picture, region counting.
- G. Cybenko, *Math. Control Signals Systems*, 1989; K. Hornik, *Neural Networks*, 1991; M. Leshno et al., *Neural Networks*, 1993 (non-polynomial activations).
- T. Zaslavsky, *Facing up to Arrangements*, AMS, 1975 — the region count.
- All plots generated by `src/build_all.py` (NumPy, Matplotlib, SciencePlots); one module per figure under `src/figures/`.

Questions?